

Testing in Python3

Unit Testing Fundamentals

- Unit testing focuses on testing a small piece of code such as a function, method, class, or module.
- Automated tests run without manual intervention and report either **pass** or **fail**.
- Strict unit tests avoid external dependencies like databases, file systems, and networks.
- Tests help verify that code behaves as expected.
- They provide confidence when modifying existing code.
- Unit testing is an essential part of professional software development.

Example

```
def add(a, b):  
    return a + b
```

```
assert add(2, 3) == 5
```

Unit Testing Vocabulary

Unit Under Test (UUT)

- The specific piece of code being tested.
- Usually a function, class, or module.
- Every test focuses on one unit.
- Helps isolate bugs quickly.
- Makes debugging easier.
- Improves code quality.

Test Case

- A single test scenario.
- Contains input, execution, and expected output.
- Designed by a developer.
- Verifies one behavior.
- Should have a meaningful name.
- Reports pass or fail.

Example

```
def test_lookup_by_name():  
    phonebook.add("Bob", "12345")  
    assert phonebook.lookup("Bob") == "12345"
```

Test Runner

- Software that executes tests.
- Collects and reports results.
- Runs individual tests or entire test suites.
- Can run from terminal or IDE.
- Examples include unittest and pytest.
- Displays passed and failed tests.

Example Output:

```
Ran 7 tests in 0.000s  
OK
```

Test Suite

- Collection of related test cases.
- Executes multiple tests together.
- Helps organize testing.

- Makes regression testing easier.
- Can contain hundreds of tests.
- Managed by a test runner.

Test Fixture

- Setup required before running tests.
- Creates common test data.
- Reduces duplication.
- Cleanup can occur afterward.
- Implemented using setUp() and tearDown().
- Used repeatedly by test cases.

Example

```
class PhonebookTest(unittest.TestCase):
```

```
    def setUp(self):  
        self.phonebook = Phonebook()
```

```
    def tearDown(self):  
        pass
```

Test Case Design

Arrange – Act – Assert Pattern

- Standard structure for writing tests.
- Arrange prepares test data.
- Act executes the functionality.
- Assert checks expected results.
- Makes tests easier to read.
- Encourages better organization.

Example

```
def test_lookup_by_name():
    phonebook.add("Bob", "12345")    # Arrange

    number = phonebook.lookup("Bob") # Act

    assert number == "12345"         # Assert
```

unittest Framework

Features of unittest

- Python's built-in testing framework.
- Inspired by JUnit.
- Uses classes for organizing tests.
- Supports assertions and fixtures.
- Included in the standard library.
- Suitable for small and large projects.

Example

```
import unittest

class PhonebookTest(unittest.TestCase):

    def setUp(self):
        self.phonebook = Phonebook()

    def test_lookup_by_name(self):
        self.phonebook.add("Bob", "12345")

        number = self.phonebook.lookup("Bob")

        self.assertEqual(number, "12345")
```

Common unittest Assertions

Frequently Used Assertions

- `assertEqual()` compares two values.
- `assertTrue()` checks if value is `True`.
- `assertFalse()` checks if value is `False`.
- `assertRaises()` checks exceptions.
- `assertIn()` verifies membership.
- Helps clearly express expectations.

Examples

```
self.assertEqual(number, "12345")
```

```
self.assertTrue(phonebook.is_consistent())
```

```
self.assertFalse(phonebook.is_consistent())
```

```
with self.assertRaises(KeyError):  
    phonebook.lookup("missing")
```

```
self.assertIn("Bob", phonebook.names())
```

Pytest

What is Pytest?

- Popular alternative to unittest.
- Simpler and more readable syntax.
- Uses Python's built-in `assert` statement.
- Supports fixtures and parameterized tests.
- Not included in Python standard library.
- One of the most widely used testing frameworks.

Example

```
def test_lookup_by_name():
    phonebook = Phonebook()

    phonebook.add("Bob", "12345")

    assert phonebook.lookup("Bob") == "12345"
```

Fixtures in Pytest

Why Use Fixtures?

- Share setup code between tests.
- Reduce duplication.
- Improve maintainability.
- Can depend on other fixtures.
- Created using decorators.
- Automatically injected into tests.

Example

```
import pytest

@pytest.fixture
def phonebook():
    return Phonebook()

def test_lookup_by_name(phonebook):
    phonebook.add("Bob", "12345")

    assert phonebook.lookup("Bob") == "12345"
```

Exception Testing in Pytest

Testing Errors

- Verifies expected exceptions occur.

- Prevents hidden bugs.
- Makes code more robust.
- Uses `pytest.raises()`.
- Works similarly to `assertRaises()`.
- Improves error handling coverage.

Example

```
def test_missing_name(phonebook):
```

```
    with pytest.raises(KeyError):
        phonebook.lookup("Bob")
```

Parameterized Testing

Why Parameterized Tests?

- Avoid duplicate test code.
- Run same test with multiple inputs.
- Improves coverage.
- Easier maintenance.
- Makes tests concise.
- Ideal for data-driven testing.

Example concept:

```
@pytest.mark.parametrize(
    "a,b,result",
    [
        (1, 2, 3),
        (2, 3, 5),
        (5, 5, 10)
    ]
)
```

Organizing Tests

Best Practices

- Keep tests separate from production code.
- Use meaningful test names.
- Group related tests together.
- Maintain a clear project structure.
- Run tests frequently.
- Avoid duplicated setup logic.

Skipping Tests

Skip and Conditional Skip

- Temporarily disable unfinished tests.
- Useful for work-in-progress features.
- Can skip based on conditions.
- Prevents unnecessary failures.
- Supported by pytest.
- Improves workflow.

Example

```
@pytest.mark.skip("Work In Progress")
def test_new_feature():
    pass
```

Conditional Skip

```
@pytest.mark.skipif(
    sys.version_info < (3, 6),
    reason="requires Python 3.6+"
)
def test_feature():
    pass
```


Why Developers Write Tests

Benefits of Automated Testing

- Ensures code behaves correctly.
- Detects bugs early.
- Prevents future regressions.
- Supports collaboration.
- Improves maintainability.
- Increases confidence in code changes.

Regression Testing

Purpose

- Ensures old functionality still works.
- Protects previously tested code.
- Detects unintended side effects.
- Helps during refactoring.
- Essential in large projects.
- Usually automated.

Test Driven Development (TDD)

What is TDD?

- Tests drive the development process.
- Write a failing test first.
- Implement code to pass the test.
- Refactor after success.
- Encourages better design.
- Improves maintainability.

TDD Cycle

Write Test



Make it Pass



Refactor



Repeat

Continuous Integration (CI)

What is CI?

- Automatically runs tests after code changes.
- Integrates changes frequently.
- Detects problems early.
- Supports team collaboration.
- Often uses build servers.
- Enables automated deployment.

CI Workflow

Commit Code



Build Project



Run Tests



Report Results



Deploy

Test Doubles

What are Test Doubles?

- Replace real dependencies during testing.
- Make tests faster and more predictable.
- Help isolate the unit under test.
- Reduce external dependencies.
- Useful for databases, APIs, and files.
- Improve test reliability.

Stub

Characteristics

- Returns predefined values.
- Simple implementation.
- Replaces a collaborator.
- No real logic.
- Used for controlled testing.
- Cannot fail tests on its own.

Example

class StubSensor:

```
def sample_pressure(self):  
    return 17
```

Fake

Characteristics

- Contains working logic.
- Simulates real behavior.
- Not suitable for production.
- Faster than real dependency.

- Commonly used for databases.
- Useful in integration tests.

Examples

- StringIO replacing files
- In-memory database
- Lightweight web server

Dummy

Characteristics

- Placeholder object.
- Not actually used.
- Exists only to satisfy parameters.
- Often represented by None.
- Simplest test double.
- Contains no behavior.

Example

dummy = None

Spy

Characteristics

- Records method calls.
- Tracks interactions.
- Allows later verification.
- Useful for behavior testing.
- Stores call history.
- More advanced than stubs.

Example Concept

`spy.calls`

Used to verify whether a function was called.

Mock

Characteristics

- Verifies expected interactions.
- Fails when calls are incorrect.
- Checks arguments and call counts.
- Useful for behavior testing.
- Stronger than spies.
- Common in complex systems.

Example Concept

`mock.notify.assert_called_once()`

Approval Testing

Purpose

- Compares output with an approved version.
- Useful when many assertions are needed.
- Shows differences clearly.
- Easier to maintain large outputs.
- Common for reports and receipts.
- Improves readability of failures.

Workflow:

Arrange

↓

Act

↓

Compare with Approved Output

Code Coverage

What is Code Coverage?

- Measures executed lines of code.
- Finds untested sections.
- Helps improve test completeness.
- Tracks testing progress.
- Often shown as percentages.
- Useful but not sufficient alone.

Mutation Testing

Purpose

- Intentionally introduces bugs.
- Verifies tests detect them.
- Measures test effectiveness.
- Stronger than coverage metrics.
- Resource intensive.
- Reveals weak tests.

Workflow:

Insert Bug

↓

Run Tests

↓

Tests Fail? ✓

Branch Coverage

Purpose

- Verifies both True and False paths execute.
- Improves conditional testing.
- Finds missing test scenarios.
- Complements statement coverage.
- Useful for decision-heavy code.
- Helps achieve thorough testing.

Hard-to-Test Code Strategies

Peel Strategy

- Move logic into a separate function.
- Leave difficult code outside.
- Test the extracted logic.
- Improves coverage.
- Reduces complexity.
- Common refactoring technique.

Example

```
def function_name():  
    return helper_function()
```

```
def helper_function():  
    # testable logic  
    pass
```

Slice Strategy

- Extract difficult section into new function.
- Isolate side effects.
- Improve testability.
- Makes logic reusable.
- Encourages cleaner design.
- Often combined with peel.

Monkey Patching

Purpose

- Replace dependencies at runtime.
- Useful when collaborators are hard to replace.
- Common in testing.
- Allows dependency substitution.
- Can make tests fragile.
- Should be used carefully.

Self-Initializing Fake

Characteristics

- Records real interactions once.
- replays them later.
- Useful for remote services.
- Speeds up testing.
- Avoids repeated network calls.
- Requires maintenance when APIs change.